

Database Indexes: The 10-Minute Foundation

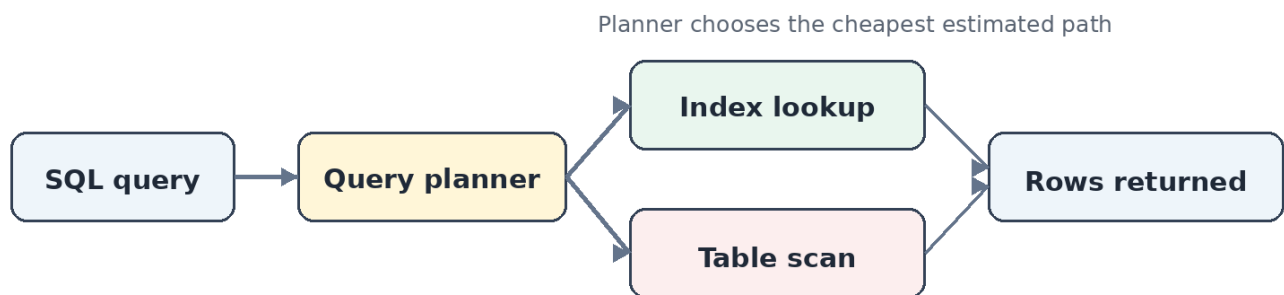
Goal: explain indexes clearly in interviews before going deeper.

One-line definition: An index is an extra data structure that helps the database locate rows faster, usually by storing selected column values in an ordered/searchable form plus a reference to the actual row.

Think of a book index, not a copy of the whole book.

- Without an index: the database may inspect many or all rows -> table/sequence scan -> roughly $O(n)$.
- With a tree index: it navigates a shallow tree to a key -> roughly $O(\log n)$, then fetches matching rows.
- The cost: extra disk/RAM, plus extra work on INSERT, UPDATE, and DELETE because the index must also be maintained.

The lifecycle of an indexed query



Term	Simple meaning	Interview phrase
Index seek / lookup	Jump to a small key range	Good when predicate is selective
Index scan	Read much of an index	Can still be cheaper than scanning full rows
Table / seq scan	Read table pages broadly	Often best when many rows match
Selectivity	How much a condition narrows rows	High selectivity usually helps indexes
Cardinality	Number of distinct values	Unique IDs = high; boolean = low

Interviewer-ready answer: “Indexes speed reads by trading extra storage and write cost for a faster access path. The optimizer decides whether using one is cheaper than scanning the table.”

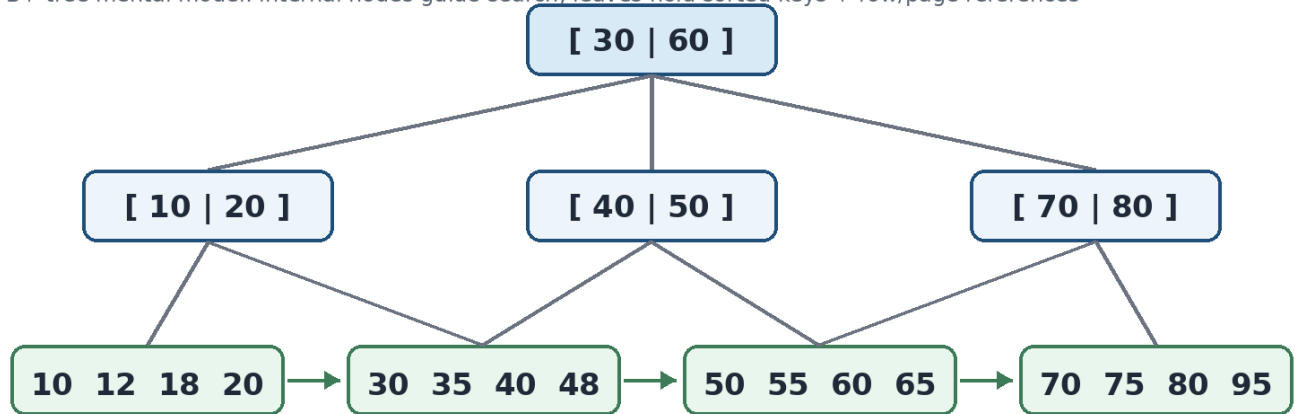
How a B+ Tree Index Works

This is the default mental model for indexes in many relational databases.

Why B+ trees?

- Keys stay sorted, so equality, range queries, ORDER BY, MIN/MAX, and prefix traversals are efficient.
- Each node stores many keys, so the tree is very shallow. A few page reads can narrow millions of rows.
- Leaf pages are linked left-to-right, which makes range scans efficient after the first matching key is found.

B+ tree mental model: internal nodes guide search; leaves hold sorted keys + row/page references



Example: WHERE user_id = 55

- Start at root -> compare 55 -> choose middle branch -> reach leaf -> find key 55 -> follow row/page reference.

Example: WHERE price BETWEEN 50 AND 80

- Navigate once to 50, then walk linked leaf pages until values exceed 80. This is why ordered indexes are strong for ranges.

What happens on writes?

Write	Index work	Possible internal effect
INSERT	Add new key entry	Page may fill and split into two pages
UPDATE indexed column	Remove old key + add new key	More random writes; possible splits
DELETE	Remove/mark entry	Cleanup/compaction may happen later

Deep but simple: A database usually reads/writes index nodes in fixed-size pages. B+ trees are designed to minimize expensive page I/O by packing many keys into each node.

PAGE 3 - MAIN INDEX TYPES

Which Index Should You Think About?

You do not need every database-specific index type for a system design interview.

Type	Best for	Main caveat
B+ tree	Equality, ranges, sorting, prefix matches	Extra maintenance; column order matters
Hash	Exact equality lookups	Usually not for ranges/sorting; DB support varies
Clustered / primary layout	Fast range + locality on clustering key	Only one physical row order/layout
Secondary / nonclustered	Alternative lookup paths	May require extra row lookup
Composite	Queries filtering by multiple columns	Leftmost column order is critical
Covering	Serve query from index alone	Larger index, more write/storage cost
Full-text / inverted	Words/tokens in text	Not a replacement for normal relational indexes
Specialized (GIN/GiST/bitmap etc.)	Arrays, JSON, geometry, analytics	Engine/workload specific

Clustered vs secondary: the useful mental model

- Clustered/primary storage: rows are physically organized close to the clustering key order, or the primary index directly contains row data, depending on the engine.
- Secondary index: stores its own key and a row locator/primary key. A lookup may become: secondary index -> row locator -> base row.

Composite indexes: column order decides what can be searched efficiently

Example: INDEX(country, city, created_at) is naturally useful for country; country+city; country+city+created_at. It usually cannot efficiently “skip” country and start directly from city.

Query pattern	Index (user_id, created_at)	Why
WHERE user_id = ?	Strong	Uses leftmost key
WHERE user_id=? AND created_at>?	Strong	Equality first, then range
WHERE created_at > ?	Usually weak	Leading user_id is missing
WHERE user_id=? ORDER BY created_at	Often strong	Index order can satisfy filter + sort

Design rule: Put common equality filters first, then range/sort columns - but always design from real query patterns, not a memorized formula.

PAGE 4 - QUERY DESIGN

When an Index Helps - and When It Does Not

The optimizer cares about cost, not the existence of an index.

Good index candidates

- Columns used often in WHERE, JOIN, ORDER BY, GROUP BY, or uniqueness constraints.
- Predicates that match a small fraction of rows (high selectivity).
- Foreign-key join columns in workloads that frequently join parent/child records.
- Stable, repeated query patterns where latency matters enough to justify write/storage overhead.

Common reasons an index is ignored or less useful

Pattern	Why it hurts	Typical fix
Low selectivity, e.g. active=true on 90% of rows	Too many row fetches; scan may be cheaper	Partial index / different access path
Function on indexed column: LOWER(name)	Normal index may not match expression	Functional/expression index if supported
Implicit cast / mismatched types	Planner may not use index cleanly	Align data and parameter types
LIKE '%abc'	Leading wildcard breaks ordered prefix search	Full-text/trigram/specialized index
Wrong composite order	Cannot efficiently reach desired key prefix	Reorder or add workload-specific index
Many rows requested	Random lookups cost more than sequential scan	Let optimizer scan; avoid forcing index

Covering index: removing the second lookup

Example query: SELECT status, created_at FROM orders WHERE user_id = ?; If the index contains user_id plus the returned columns, the engine may answer from the index without touching the base table (“index-only scan”).

Partial / filtered index

- Index only the subset that matters, e.g. orders WHERE status = 'PENDING'. Smaller index, less write cost, great when the subset is frequently queried.

A practical system-design question

“Should every WHERE column be indexed?” No. Indexes should match important query patterns. Too many indexes increase writes, storage, memory pressure, vacuum/maintenance work, and optimizer complexity.

The Problems Indexes Create

Knowing when not to add one is senior-level signal.

Problem	What is happening	Impact
Write amplification	Every write may update several indexes	Slower writes + more I/O
Disk/RAM usage	Indexes are separate structures/pages	Bigger backups + cache pressure
Page splits / fragmentation	Ordered pages fill and must split/reorganize	Extra writes, lower locality
Duplicate/redundant indexes	Two indexes serve nearly same queries	Cost with little read benefit
Stale statistics	Optimizer misestimates row counts/selectivity	Bad plan even when index exists
Hot keys / contention	Many operations touch same key/page range	Latch/lock/cache contention

How to debug a slow query

1. Run EXPLAIN / EXPLAIN ANALYZE (or your engine equivalent).
2. Check: table scan vs index scan/seek, estimated rows vs actual rows, sort/hash operations, row lookups.
3. Ask whether the filter is selective and whether the index column order matches the predicate.
4. Check data types, functions/casts, wildcard patterns, and stale statistics.
5. Measure before/after. An index that looks correct can still be slower for the real data distribution.

Useful complexity intuition (not a promise)

Operation	No useful index	B+ tree intuition
Equality search	$O(n)$	$O(\log n) + \text{matching row fetches}$
Range search	$O(n)$	$O(\log n + k)$, where k = matches
Insert	Base-row write	Base-row write + tree maintenance

System-design trade-off sentence: “For a read-heavy service I may accept extra indexes. For a write-heavy ingestion path, I keep indexes minimal and add only those justified by read/query SLAs.”

Fast Answers to Common Index Questions

Use these as compact speaking patterns, not scripts to memorize word-for-word.

Q: Why not index every column?

A: Every index consumes storage and must be updated on writes. Extra indexes can reduce write throughput and may provide no useful query benefit.

Q: Why can a database ignore my index?

A: The optimizer may estimate that a scan is cheaper, especially if many rows match, statistics suggest low selectivity, or the query shape does not match the index.

Q: B-tree vs hash index?

A: B+ tree keeps order, so it supports equality, ranges, and ordered scans. Hash is naturally suited to exact equality and does not provide ordered range traversal.

Q: What is the leftmost-prefix rule?

A: For an index on (A,B,C), efficient navigation normally starts with A, then optionally B and C. A query only on B cannot usually jump directly to the right B range.

Q: What is a covering index?

A: An index containing the filter/join key plus all columns needed by the query, allowing the engine to avoid fetching the base row in favorable cases.

Q: What is selectivity?

A: How strongly a predicate reduces candidate rows. A value matching 0.1% of rows is far more selective than one matching 80%.

Q: What happens during a B+ tree page split?

A: When a page is full, entries are redistributed into new pages and parent pointers/separators are updated. This preserves order but adds write/I/O cost.

Q: How do you choose an index in system design?

A: Start from critical query patterns and SLAs, examine filters/joins/sorts and data distribution, choose the smallest index that serves them, then verify using real query plans and measurements.

Final 20-second summary: “An index is a separate access structure, commonly a B+ tree, that trades write/storage cost for faster reads. Good indexes match selective query patterns and column order. The optimizer still chooses the cheapest plan, so I verify with EXPLAIN and real measurements.”